

TERRAFORM TASK - 02

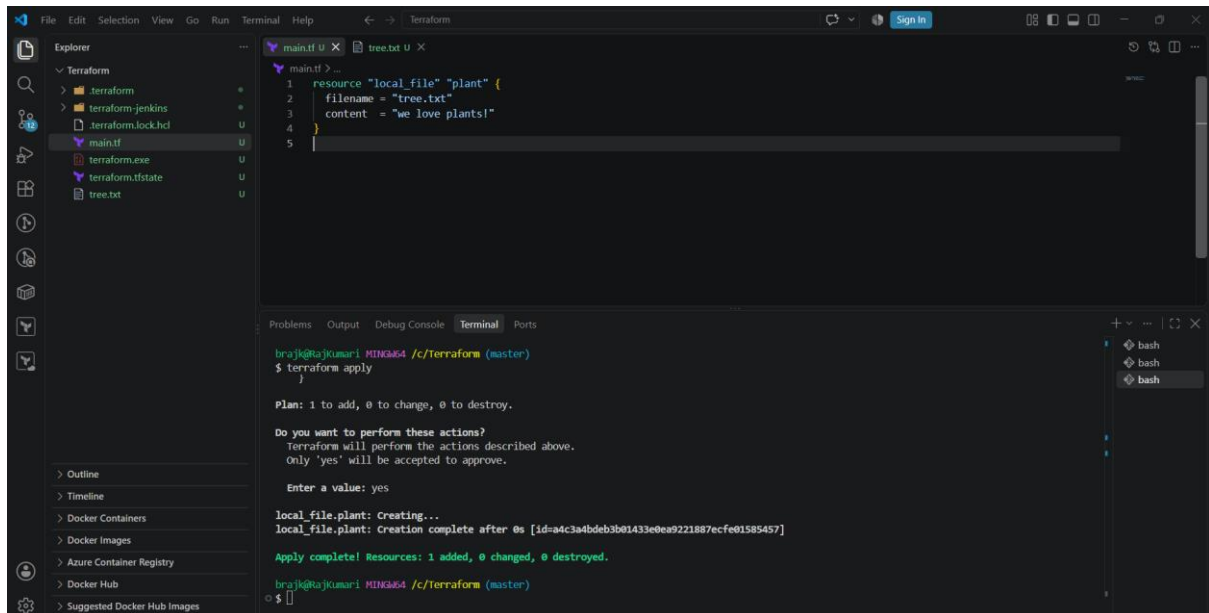
1. Execute all the templates shown in the video.

File: main.tf

main.tf — Main Infrastructure Configuration

Purpose: main.tf contains the **actual infrastructure resources** that Terraform needs to create, modify, or delete.

Command: `$ terraform init`, `$ terraform fmt`, `$ terraform apply`



The screenshot shows a Visual Studio Code editor with a file explorer on the left displaying a project structure for Terraform. The main editor window shows the content of `main.tf`:

```
1 resource "local_file" "plant" {
2   filename = "tree.txt"
3   content  = "we love plants!"
4 }
5
```

Below the editor, a terminal window is open, showing the execution of the `terraform apply` command. The output indicates that the `local_file.plant` resource is being created and the operation is successful.

```
brajk@kajkumar: MINGW64 /c/Terraform (master)
$ terraform apply

Plan: 1 to add, 0 to change, 0 to destroy.

Do you want to perform these actions?
Terraform will perform the actions described above.
only 'yes' will be accepted to approve.

Enter a value: yes

local_file.plant: Creating...
local_file.plant: Creation complete after 0s [id=a4c3a4bdeb3b01433e0ea9221887ecfe01585457]

Apply complete! Resources: 1 added, 0 changed, 0 destroyed.

brajk@kajkumar: MINGW64 /c/Terraform (master)
$
```

terraform.tfstate — Terraform State File

This is one of the **most important Terraform files**.

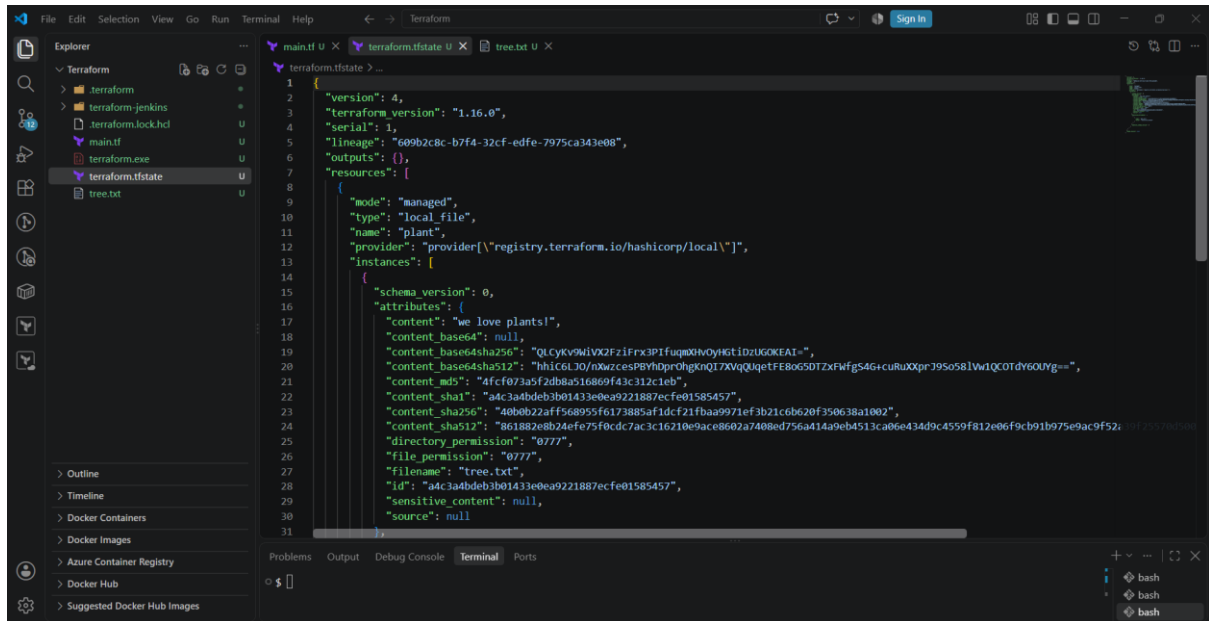
Purpose: terraform.tfstate stores Terraform's knowledge about the **infrastructure it manages**.

Terraform uses the state file to understand:

- What resources it created
- Resource IDs
- Current known configuration

- Relationships between resources
- Attributes of managed resources

File: terraform.tfstate



.terraform.lock.hcl — Provider Dependency Lock File

This file is created when you run:

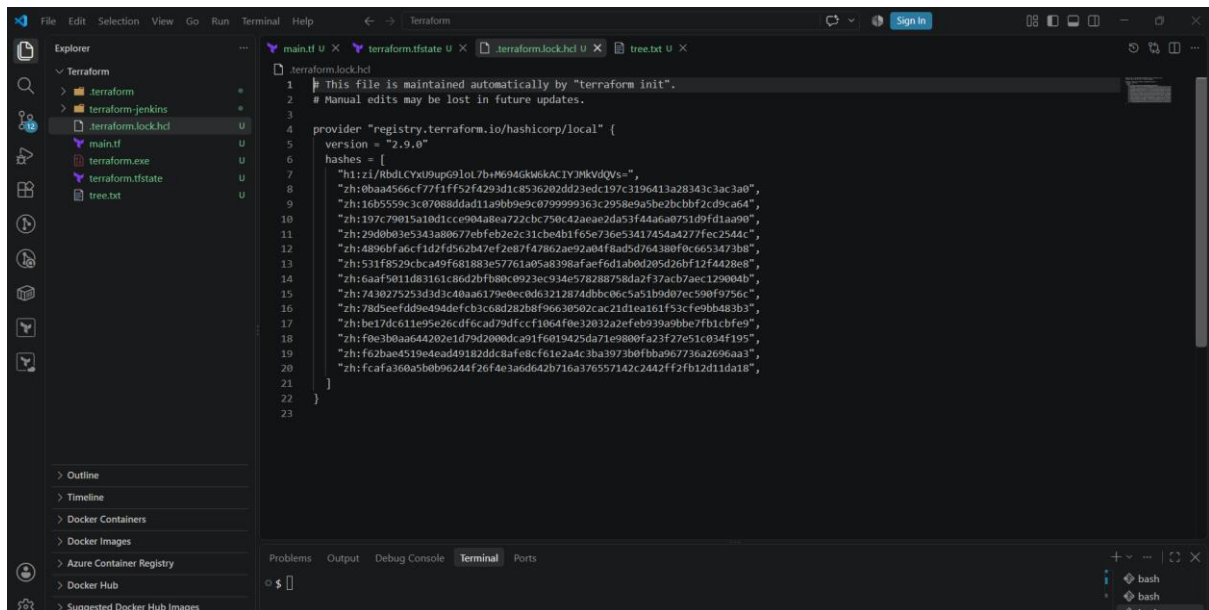
terraform init

It records the **provider versions and checksums** Terraform selected.

For example:

.terraform.lock.hcl

File: .terraform.lock.hcl



variables.tf — Input Variables

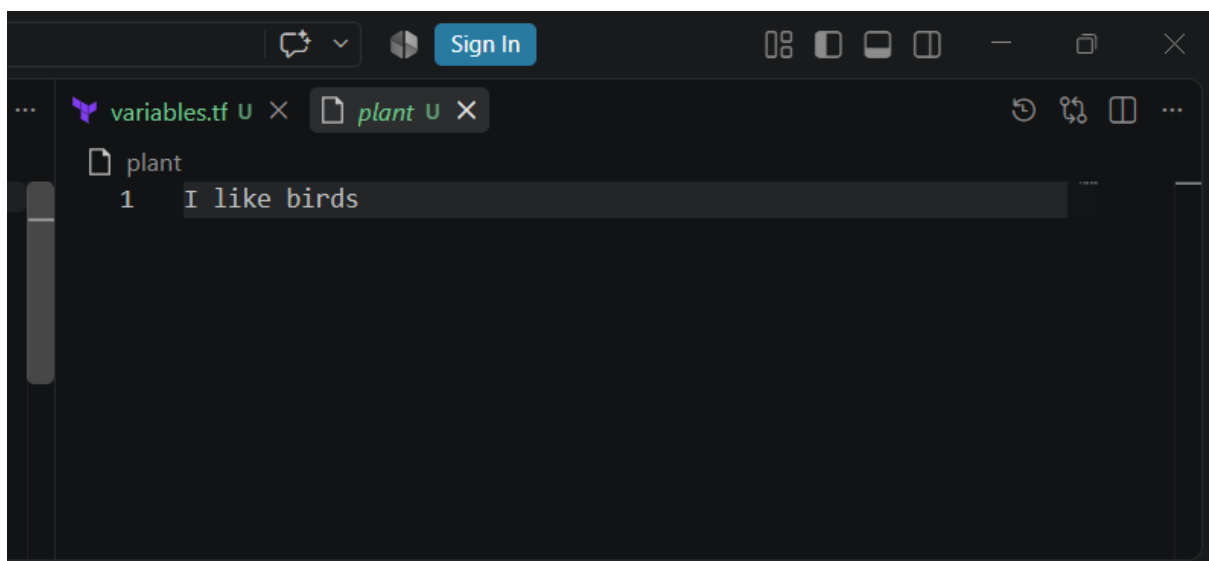
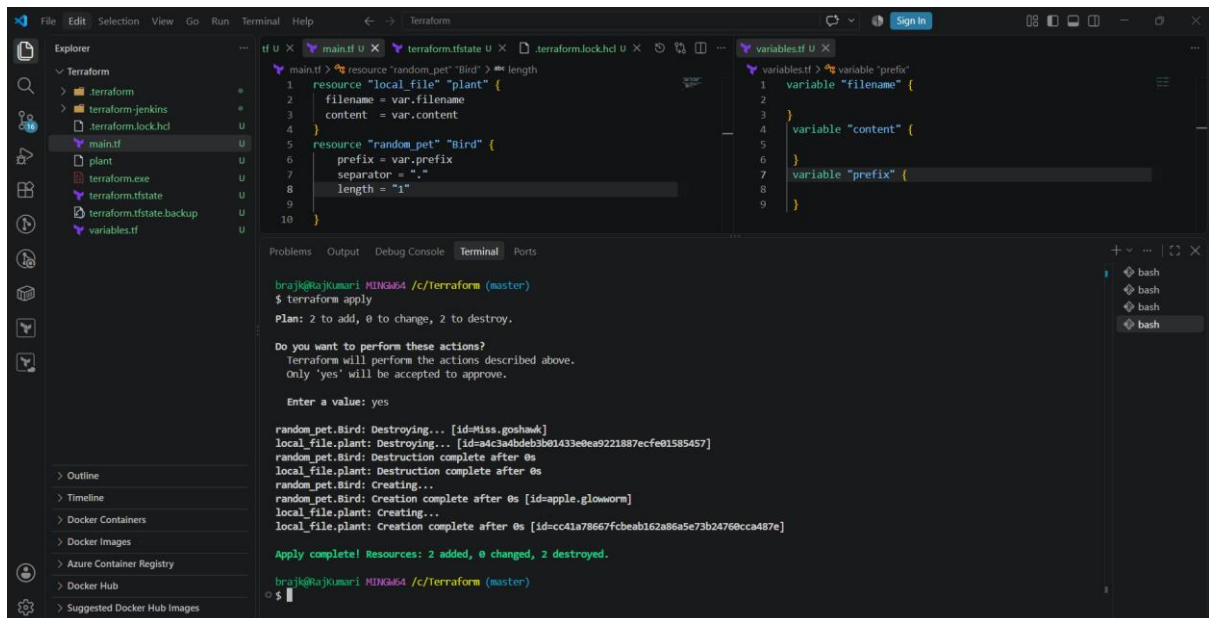
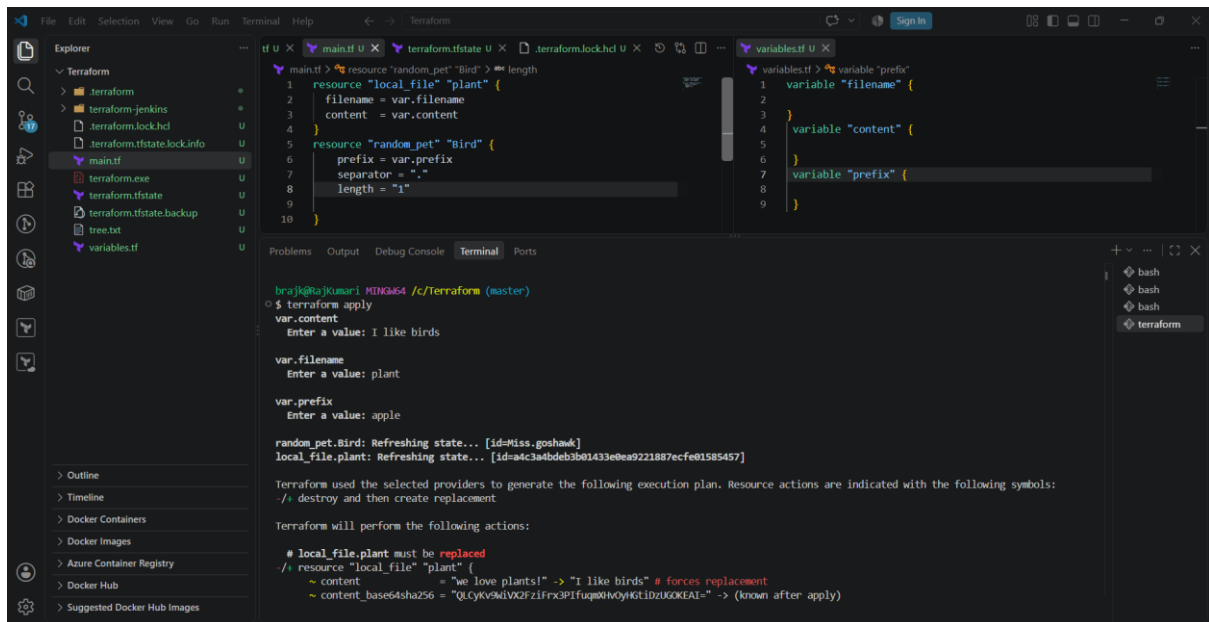
Purpose: variables.tf defines **input values** that can be passed into your Terraform configuration.

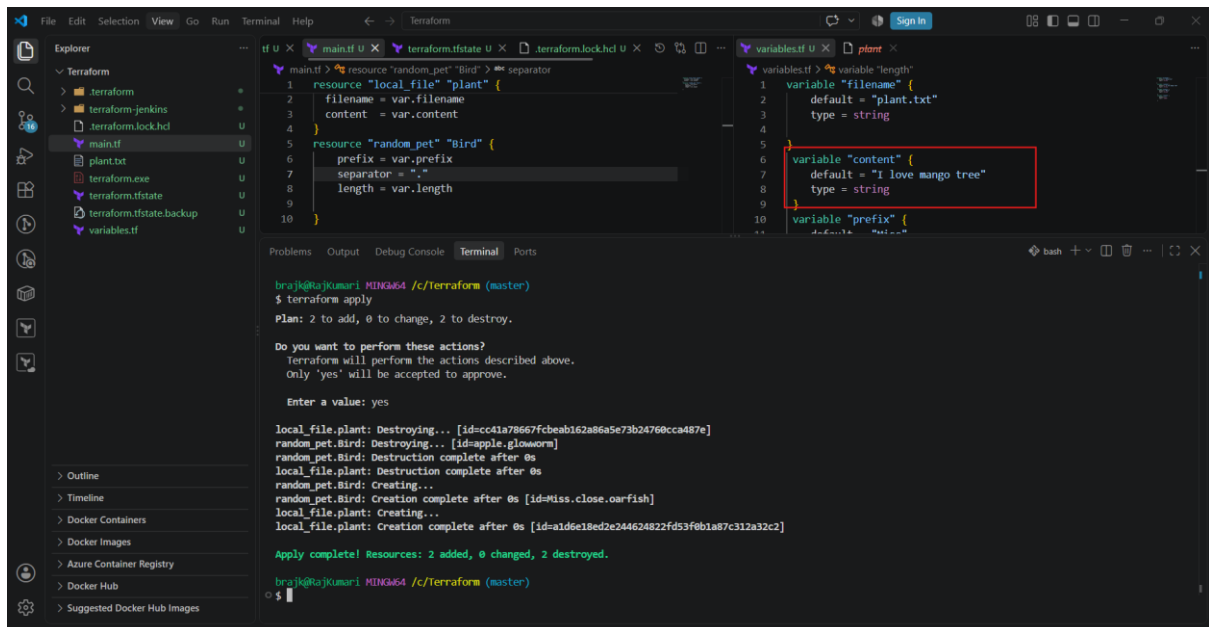
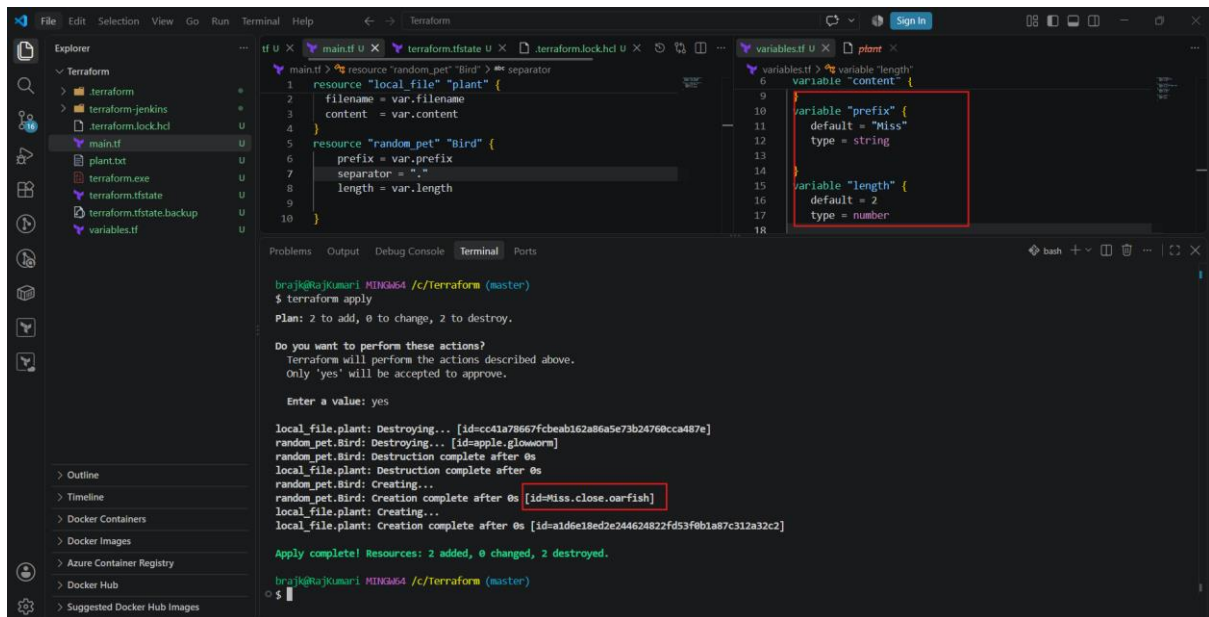
I am running terraform apply. Terraform reads my configuration files and asks for the values of input variables because no default values are defined.

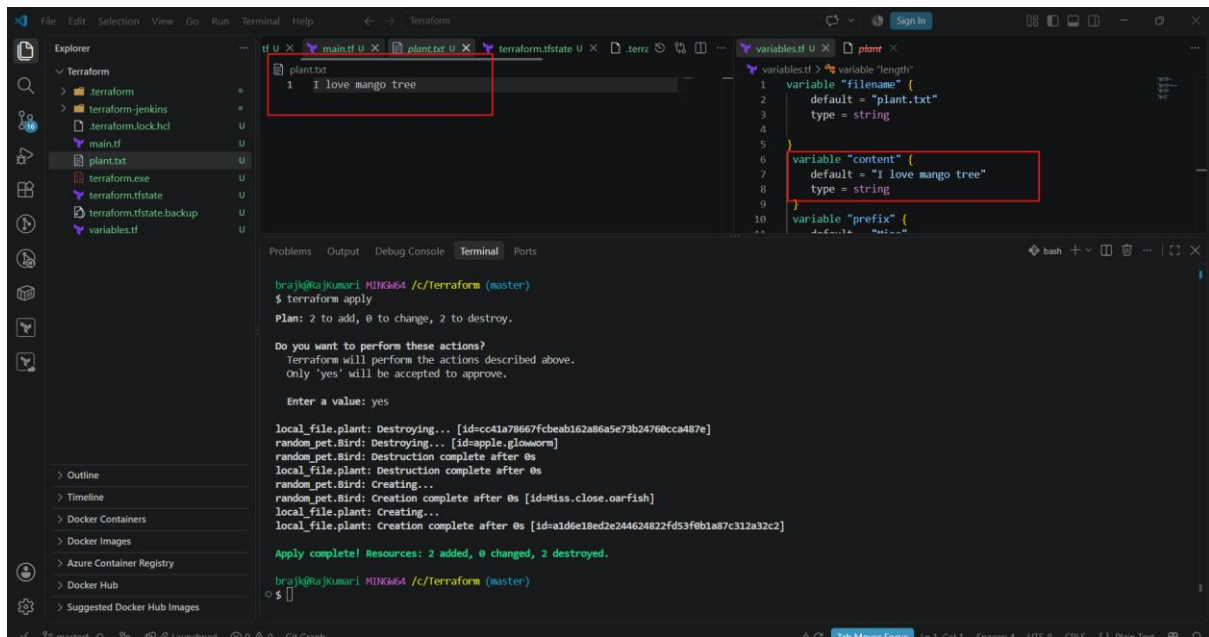
After receiving the values, Terraform compares the desired configuration with the current state.

It detects that the local_file.plant resource needs to be replaced, which is represented by -/+. Terraform will destroy the old resource and create a new one with the updated values.

File: variables.tf

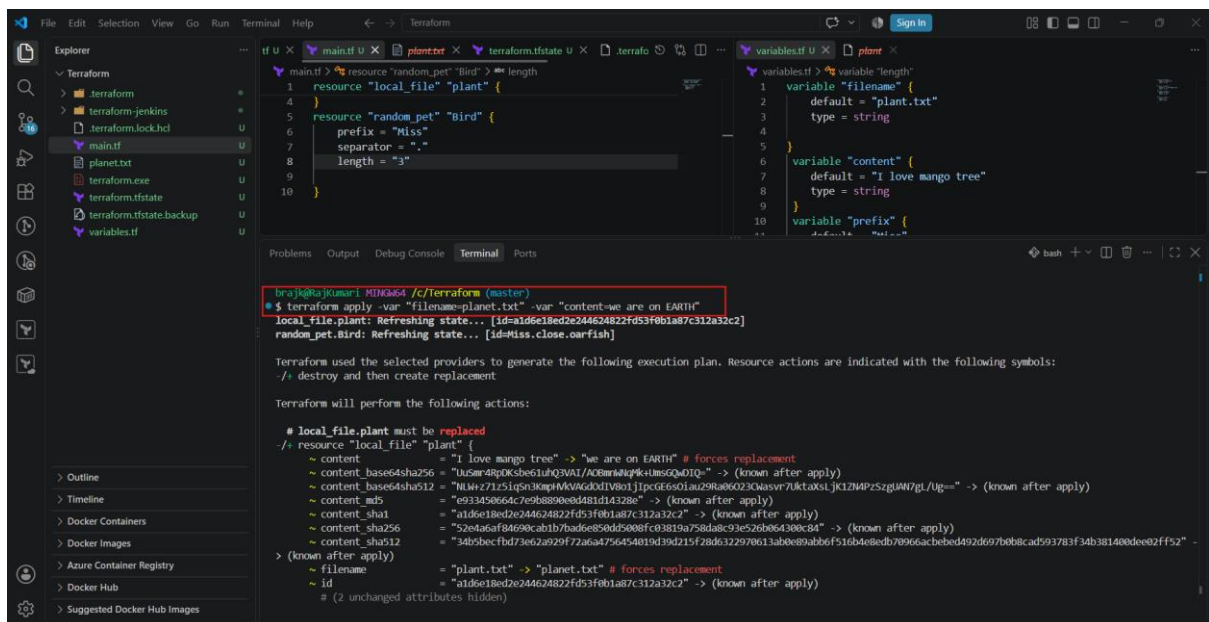


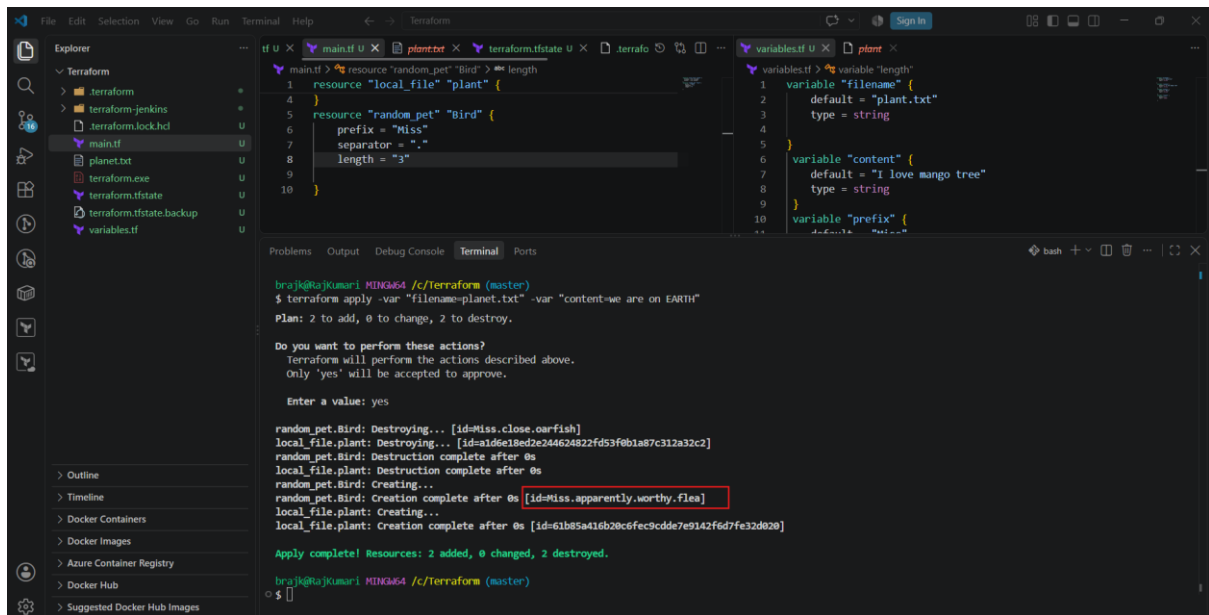




We can provide the values through terminal also with the help of “-var” command

Command: `$ terraform apply -var "filename=planet.txt" -var "content=we are on EARTH"`





```
tf u x main.tf x planet.txt x terraform.tfstate u x terraform.exe x terraform.tfstate.backup x variables.tf x
1 resource "random_pet" "bird" {
2   length
3
4   resource "random_pet" "bird" {
5     prefix = "Miss"
6     separator = "."
7     length = 3
8   }
9
10 }

variables.tf x
1 variable "length" {
2   type = string
3 }
4
5 variable "filename" {
6   default = "plant.txt"
7   type = string
8 }
9
10 variable "content" {
11   default = "I love mango tree"
12   type = string
13 }
14
15 variable "prefix" {
16   default = "Miss"
17 }

Problems Output Debug Console Terminal Ports
bash + - - - - -

brajk@rajikumar: ~/c/terraform (master)
$ terraform apply -var "filename=planet.txt" -var "content=we are on EARTH"
Plan: 2 to add, 0 to change, 2 to destroy.

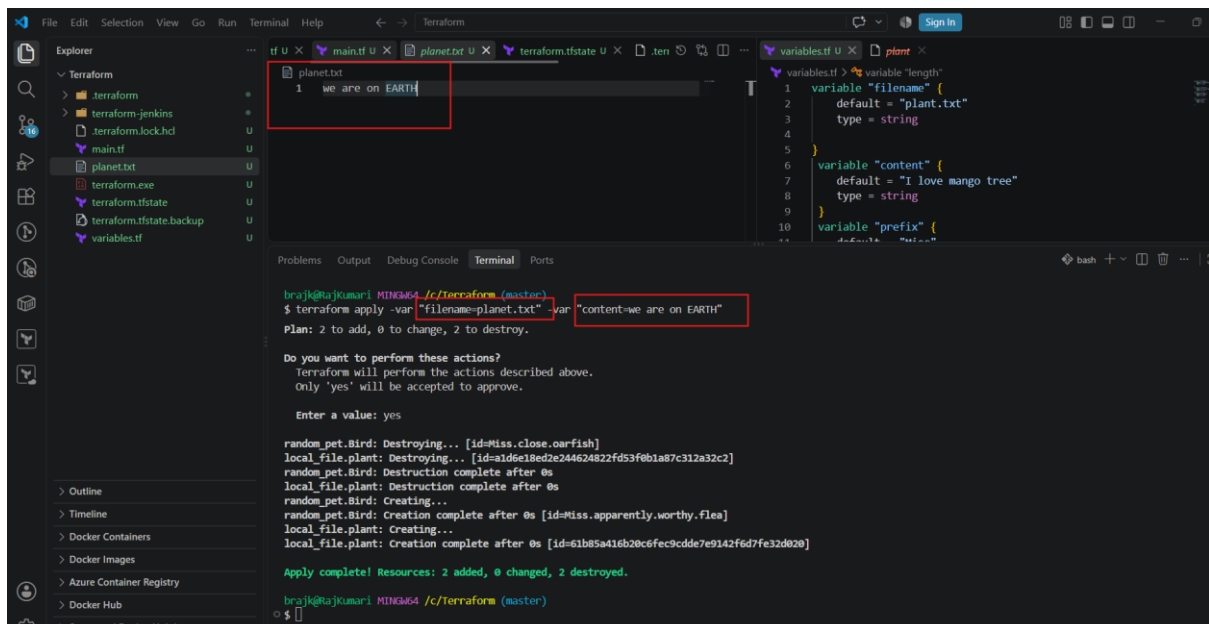
Do you want to perform these actions?
Terraform will perform the actions described above.
Only 'yes' will be accepted to approve.

Enter a value: yes

random_pet.bird: Destroying... [id=Miss.close.oarfish]
local_file.plant: Destroying... [id=a1d6e18ed2e244624822fd53f0b1a87c312a32c2]
random_pet.bird: Destruction complete after 0s
local_file.plant: Destruction complete after 0s
random_pet.bird: Creating...
random_pet.bird: Creation complete after 0s [id=Miss.apparently.worthy.flea]
local_file.plant: Creating...
local_file.plant: Creation complete after 0s [id=61b85a416b20c6fec9cdd7e9142fed7fe32d020]

Apply complete! Resources: 2 added, 0 changed, 2 destroyed.

brajk@rajikumar: ~/c/terraform (master)
$
```



```
tf u x main.tf x planet.txt x terraform.tfstate u x terraform.exe x terraform.tfstate.backup x variables.tf x
1 we are on EARTH

variables.tf x
1 variable "length" {
2   type = string
3 }
4
5 variable "filename" {
6   default = "plant.txt"
7   type = string
8 }
9
10 variable "content" {
11   default = "I love mango tree"
12   type = string
13 }
14
15 variable "prefix" {
16   default = "Miss"
17 }

Problems Output Debug Console Terminal Ports
bash + - - - - -

brajk@rajikumar: ~/c/terraform (master)
$ terraform apply -var "filename=planet.txt" -var "content=we are on EARTH"
Plan: 2 to add, 0 to change, 2 to destroy.

Do you want to perform these actions?
Terraform will perform the actions described above.
Only 'yes' will be accepted to approve.

Enter a value: yes

random_pet.bird: Destroying... [id=Miss.close.oarfish]
local_file.plant: Destroying... [id=a1d6e18ed2e244624822fd53f0b1a87c312a32c2]
random_pet.bird: Destruction complete after 0s
local_file.plant: Destruction complete after 0s
random_pet.bird: Creating...
random_pet.bird: Creation complete after 0s [id=Miss.apparently.worthy.flea]
local_file.plant: Creating...
local_file.plant: Creation complete after 0s [id=61b85a416b20c6fec9cdd7e9142fed7fe32d020]

Apply complete! Resources: 2 added, 0 changed, 2 destroyed.

brajk@rajikumar: ~/c/terraform (master)
$
```

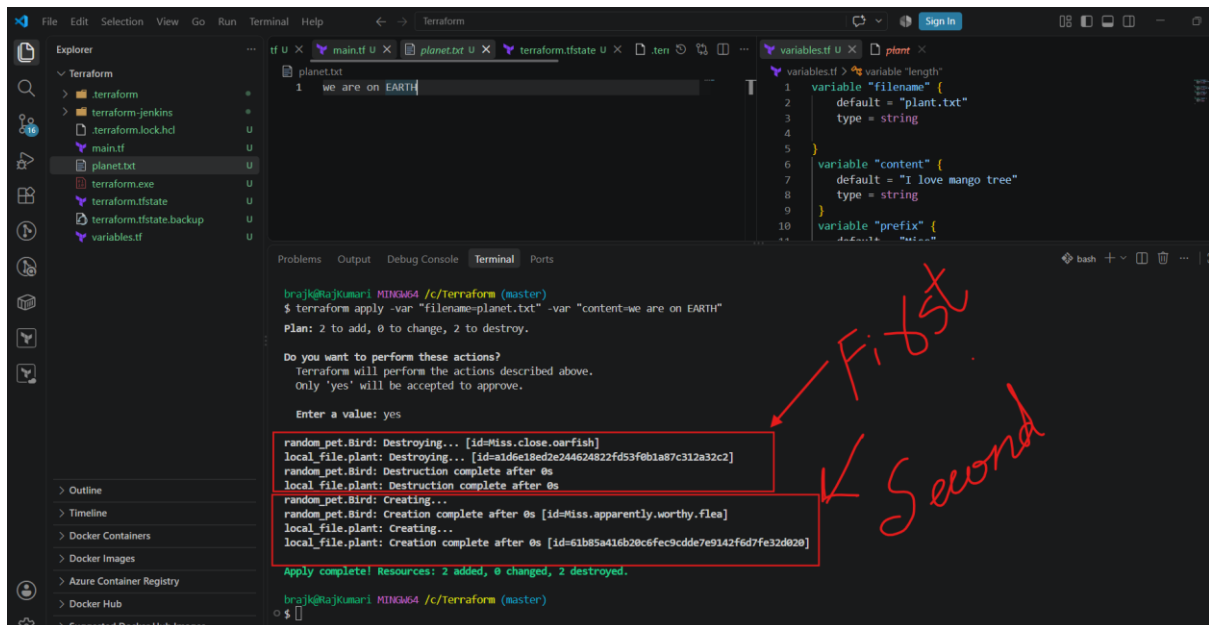
Terraform Lifecycle Rules

Terraform **lifecycle rules** control how Terraform creates, updates, and destroys resources. They are written inside a lifecycle block within a resource.

There are **three commonly used lifecycle rules**:

1. create_before_destroy
2. prevent_destroy
3. ignore_changes

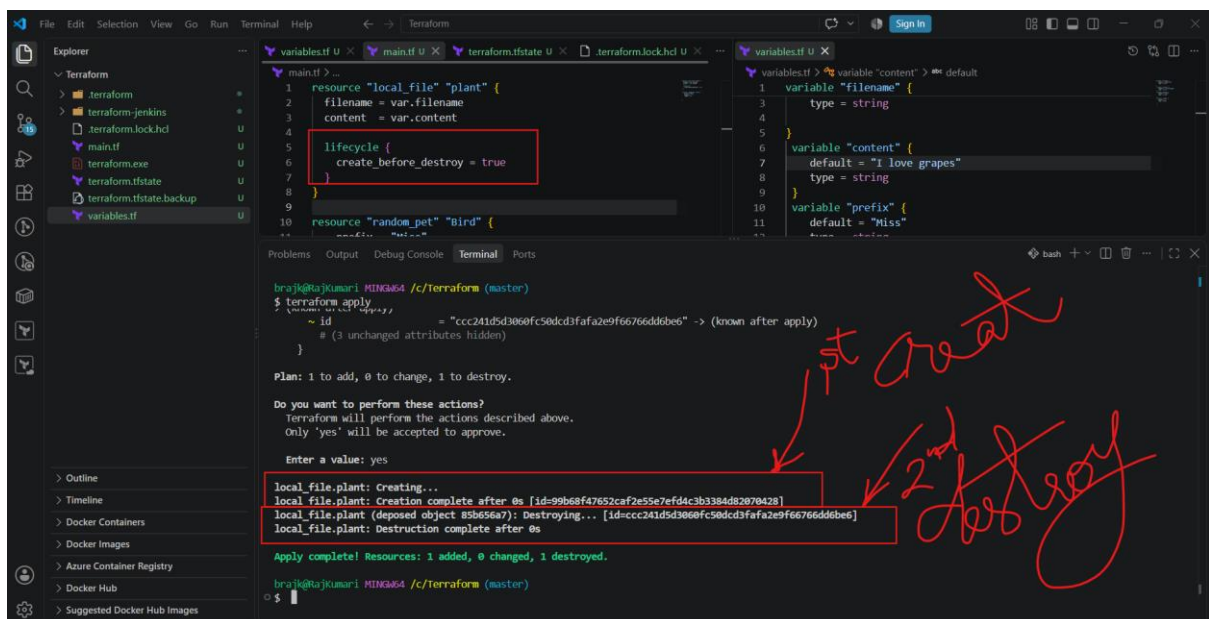
Here, the below image by default first, terraform will Destroy the previous old **template** resources(infrastructure) and later terraform will create the new template with new resources(infrastructure)



1. create_before_destroy

Normally, Terraform may **destroy** the old resource first and then create the new one.

With `create_before_destroy = true`, Terraform creates the **new resource** first, then destroys the old resource



2. prevent_destroy

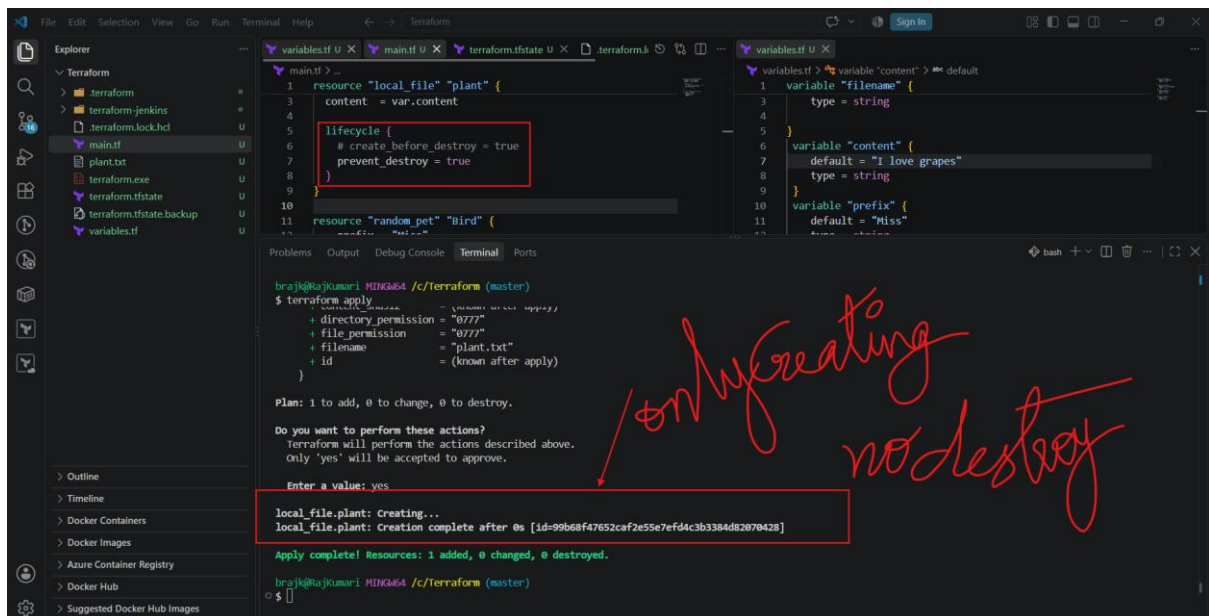
This prevents Terraform from destroying a resource.

When to use

Useful for critical resources such as:

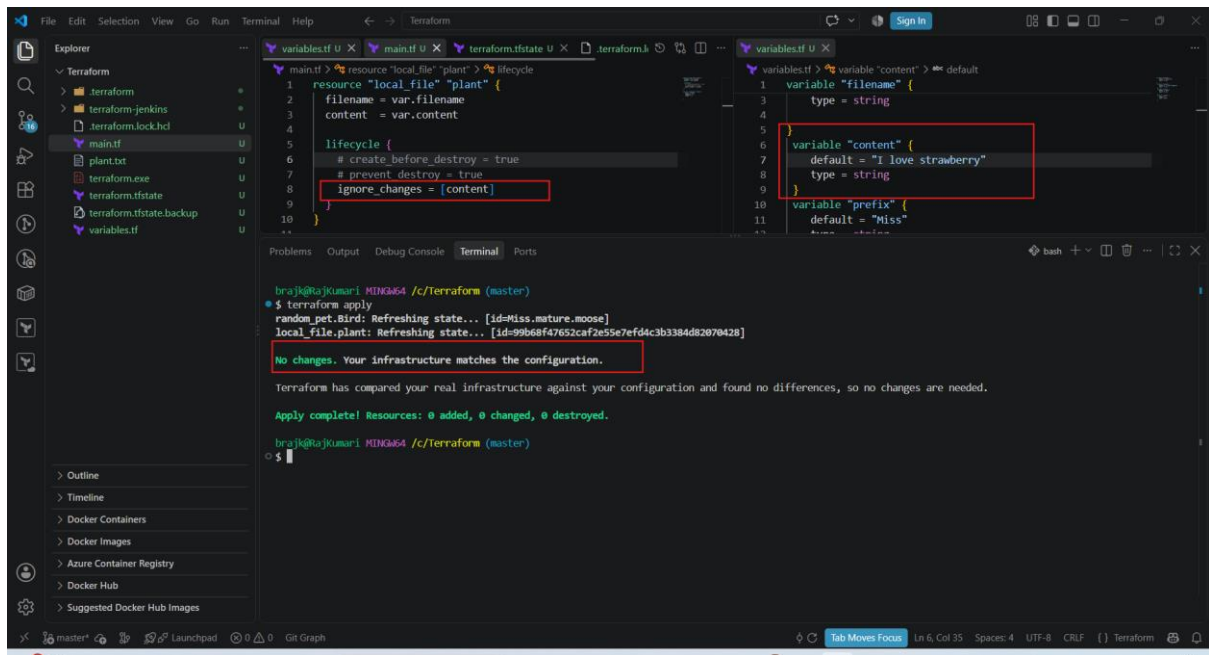
- Production databases
- Important S3 buckets
- Critical infrastructure

Important: `prevent_destroy` is a Terraform configuration safeguard. It doesn't protect a resource from deletion by someone using the cloud provider's console/API directly.



3. ignore_changes

This tells Terraform to **ignore changes to specified attributes** when checking for updates.



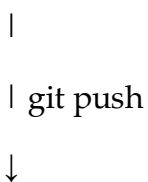
Terraform lifecycle rules control how Terraform handles resource creation, modification, and destruction. The main lifecycle arguments are **create_before_destroy**, **prevent_destroy**, and **ignore_changes**. **create_before_destroy** creates the new resource before deleting the old one, **prevent_destroy** prevents Terraform from destroying a resource, and **ignore_changes** tells Terraform to ignore changes to specified attributes.

2.Integrate Terraform in Jenkins using the Terraform plugin.

Terraform + Jenkins Integration

Architecture

GitHub



Jenkins

|

| Terraform Plugin

↓

Terraform

|

| AWS Provider

↓

AWS Infrastructure

1. Install Terraform on Jenkins Server

First check whether Terraform is installed:

`terraform version`

If you get:

`terraform: command not found`

install Terraform.

For Amazon Linux 2023, for example:

`sudo dnf install -y yum-utils`

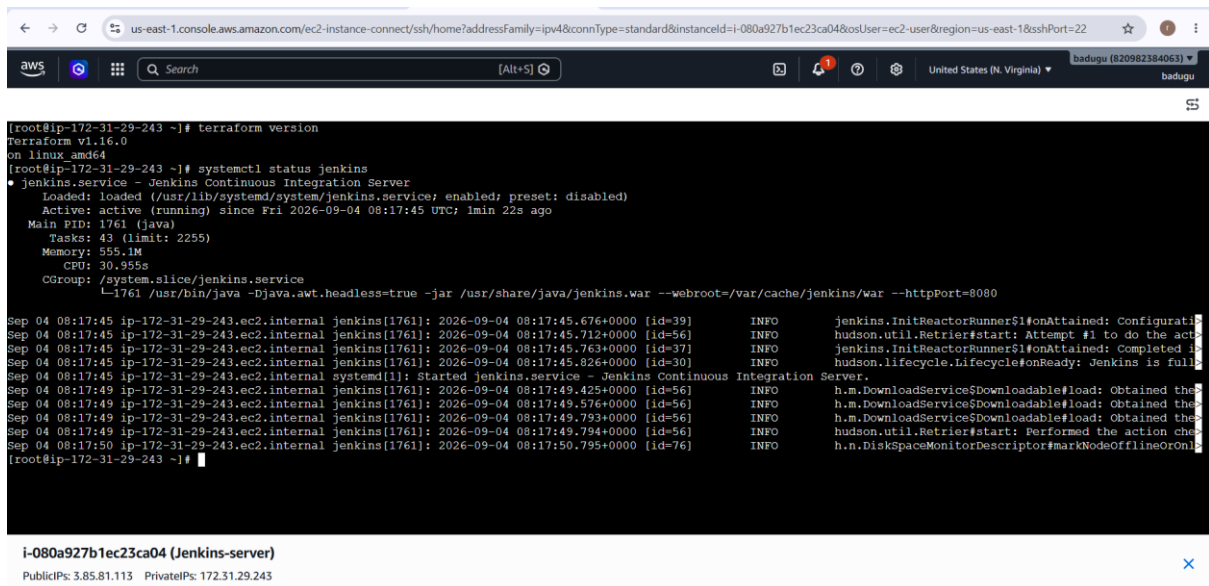
Then install Terraform using the HashiCorp repository or the official Terraform binary.

After installation:

`terraform version`

You should see something similar to:

Terraform v1.x.x



The screenshot shows an AWS console terminal window with the following content:

```
[root@ip-172-31-29-243 ~]# terraform version
Terraform v1.16.0
on linux_amd64
[root@ip-172-31-29-243 ~]# systemctl status jenkins
● jenkins.service - Jenkins Continuous Integration Server
   Loaded: loaded (/usr/lib/systemd/system/jenkins.service; enabled; preset: disabled)
   Active: active (running) since Fri 2026-09-04 08:17:45 UTC; 1min 22s ago
     Main PID: 1761 (java)
       Tasks: 43 (limit: 2255)
      Memory: 555.1M
         CPU: 30.955s
        CGroup: /system.slice/jenkins.service
                └─1761 /usr/bin/java -Djava.awt.headless=true -jar /usr/share/java/jenkins.war --webroot=/var/cache/jenkins/war --httpPort=8080

Sep 04 08:17:45 ip-172-31-29-243.ec2.internal jenkins[1761]: 2026-09-04 08:17:45.676+0000 [id=39] INFO jenkins.InitReactorRunner$1#onAttained: Configurati
Sep 04 08:17:45 ip-172-31-29-243.ec2.internal jenkins[1761]: 2026-09-04 08:17:45.712+0000 [id=56] INFO hudson.util.Retrier#start: Attempt #1 to do the act
Sep 04 08:17:45 ip-172-31-29-243.ec2.internal jenkins[1761]: 2026-09-04 08:17:45.763+0000 [id=37] INFO jenkins.InitReactorRunner$1#onAttained: Completed in
Sep 04 08:17:45 ip-172-31-29-243.ec2.internal systemd[1]: Started jenkins.service - Jenkins Continuous Integration Server.
Sep 04 08:17:45 ip-172-31-29-243.ec2.internal jenkins[1761]: 2026-09-04 08:17:45.826+0000 [id=30] INFO hudson.lifecycle.Lifecycle#onReady: Jenkins is full
Sep 04 08:17:49 ip-172-31-29-243.ec2.internal jenkins[1761]: 2026-09-04 08:17:49.425+0000 [id=56] INFO h.m.DownloadService$Downloadable#load: Obtained the
Sep 04 08:17:49 ip-172-31-29-243.ec2.internal jenkins[1761]: 2026-09-04 08:17:49.576+0000 [id=56] INFO h.m.DownloadService$Downloadable#load: Obtained the
Sep 04 08:17:49 ip-172-31-29-243.ec2.internal jenkins[1761]: 2026-09-04 08:17:49.793+0000 [id=56] INFO h.m.DownloadService$Downloadable#load: Obtained the
Sep 04 08:17:49 ip-172-31-29-243.ec2.internal jenkins[1761]: 2026-09-04 08:17:49.794+0000 [id=56] INFO hudson.util.Retrier#start: Performed the action che
Sep 04 08:17:50 ip-172-31-29-243.ec2.internal jenkins[1761]: 2026-09-04 08:17:50.795+0000 [id=76] INFO h.n.DiskSpaceMonitorDescriptor#markNodeOfflineOrOnl
```

Below the terminal output, the instance details are shown:

i-080a927b1ec23ca04 (Jenkins-server)
PublicIPs: 3.85.81.113 PrivateIPs: 172.31.29.243

2. Install Terraform Plugin in Jenkins

Go to:

Jenkins



Manage Jenkins



Plugins



Available plugins

Search for:

Terraform

Install the **Terraform plugin**.

Also make sure these commonly required plugins are available:

- Git
- Pipeline
- Credentials Binding
- Terraform

Restart Jenkins if requested.

The screenshot shows the Jenkins web interface at the URL `3.85.81.113:8080/manage/pluginManager/available`. The page title is "Jenkins / Manage Jenkins / Plugins". The "Plugins" section is active, showing a sidebar with "Updates" (2), "Available plugins", "Installed plugins", and "Advanced settings". A search bar contains the text "terrafo". To the right of the search bar are buttons for "Install", a dropdown arrow, and a refresh icon. Below the search bar is a table of available plugins:

Install	Name ↓	Released	Health
☒	Terraform 1.0.10 Build Wrappers This plugin provides a build wrapper for Terraform to launch and destroy infrastructure.	6 yr 6 mo ago	70

3. Configure Terraform in Jenkins

Go to:

Manage Jenkins



Tools

Find:

Terraform installations

Click:

Add Terraform

Give it a name:

terraform

You can configure the Terraform executable automatically or specify the installation path.

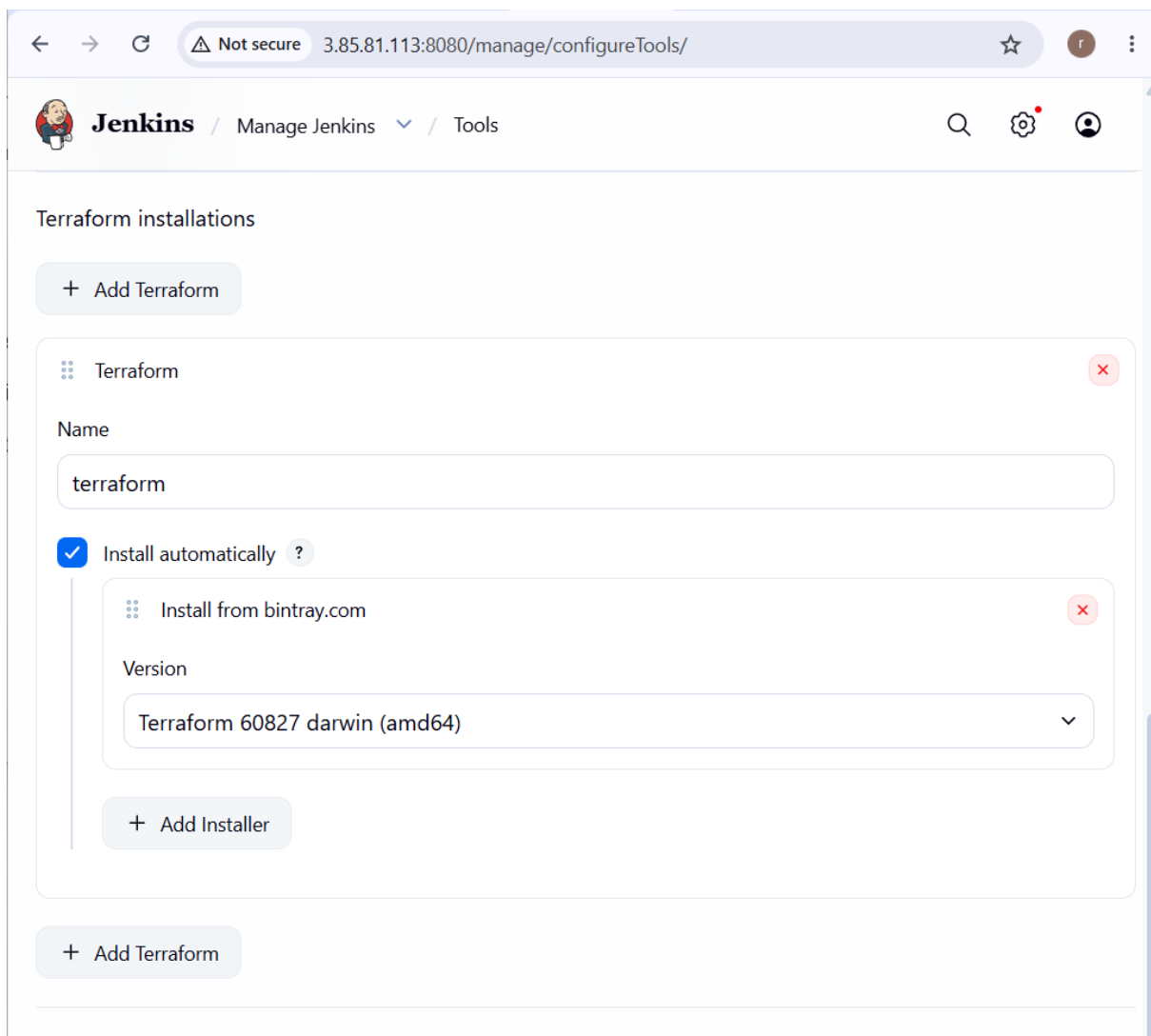
For example, if Terraform is installed at:

/usr/local/bin/terraform

use that installation.

Then click:

Save



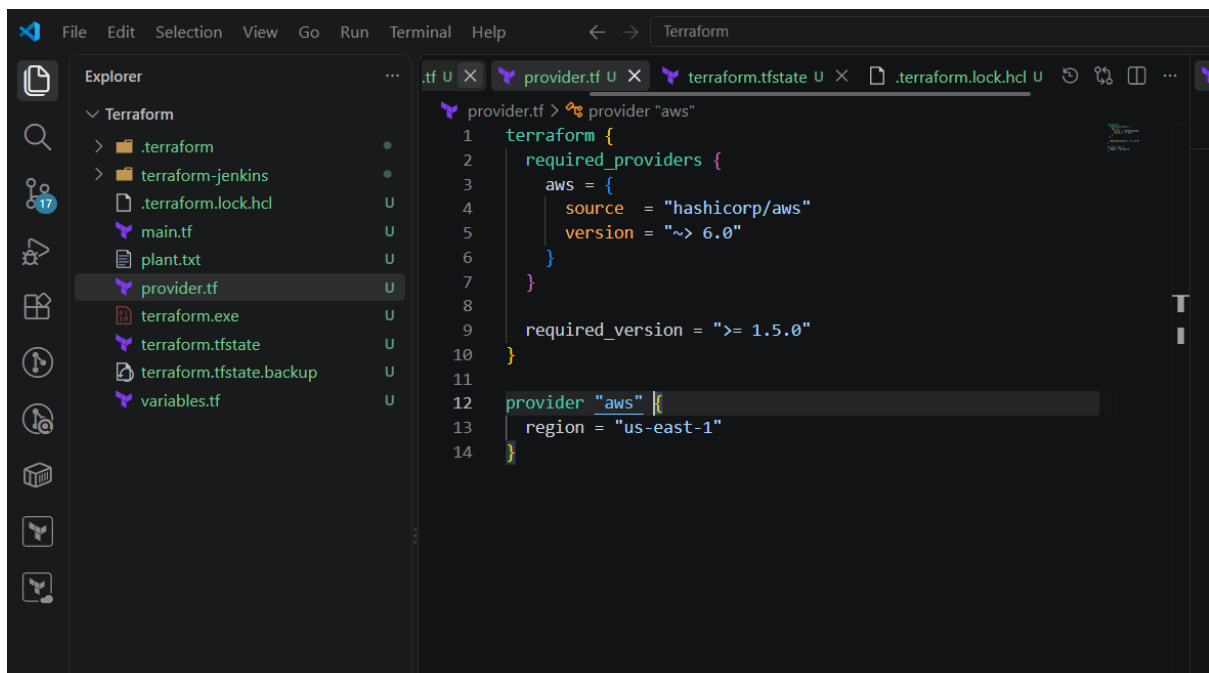
4. Create Your Terraform Project

Suppose your GitHub repository contains:

terraform-jenkins/

- └─ main.tf
- └─ provider.tf
- └─ variables.tf
- └─ outputs.tf
- └─ README.md

provider.tf



```
provider.tf > provider "aws"
1 terraform {
2   required_providers {
3     aws = {
4       source = "hashicorp/aws"
5       version = "~> 6.0"
6     }
7   }
8
9   required_version = ">= 1.5.0"
10 }
11
12 provider "aws" {
13   region = "us-east-1"
14 }
```

5. Configure AWS Credentials

There are two common approaches.

Recommended for an EC2-based Jenkins server

Attach an **IAM role** to the Jenkins EC2 instance.

For example:

Jenkins EC2

↓

IAM Role

↓

AWS permissions



Terraform



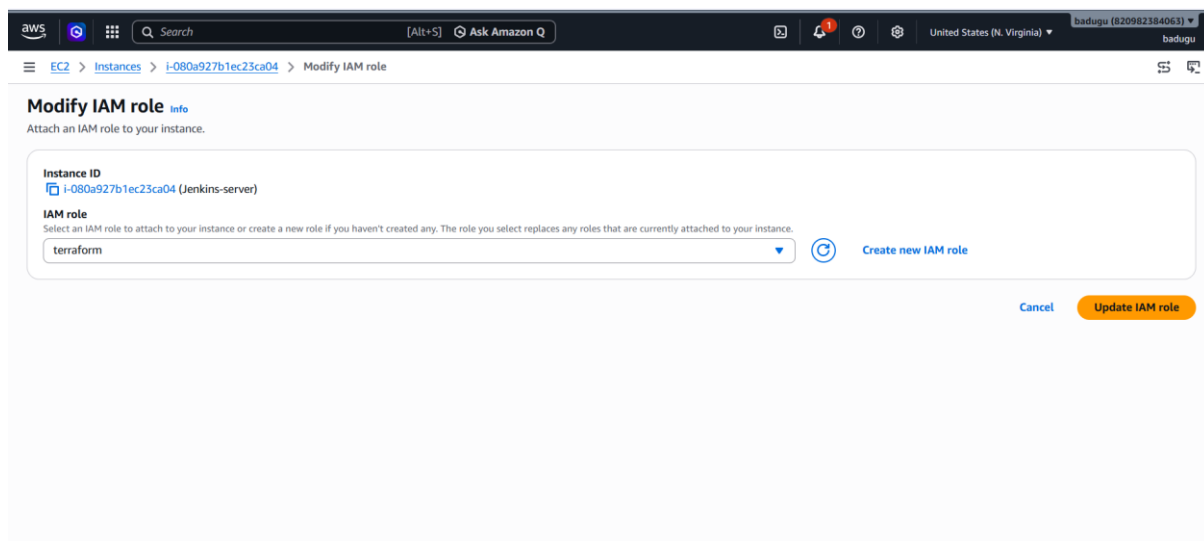
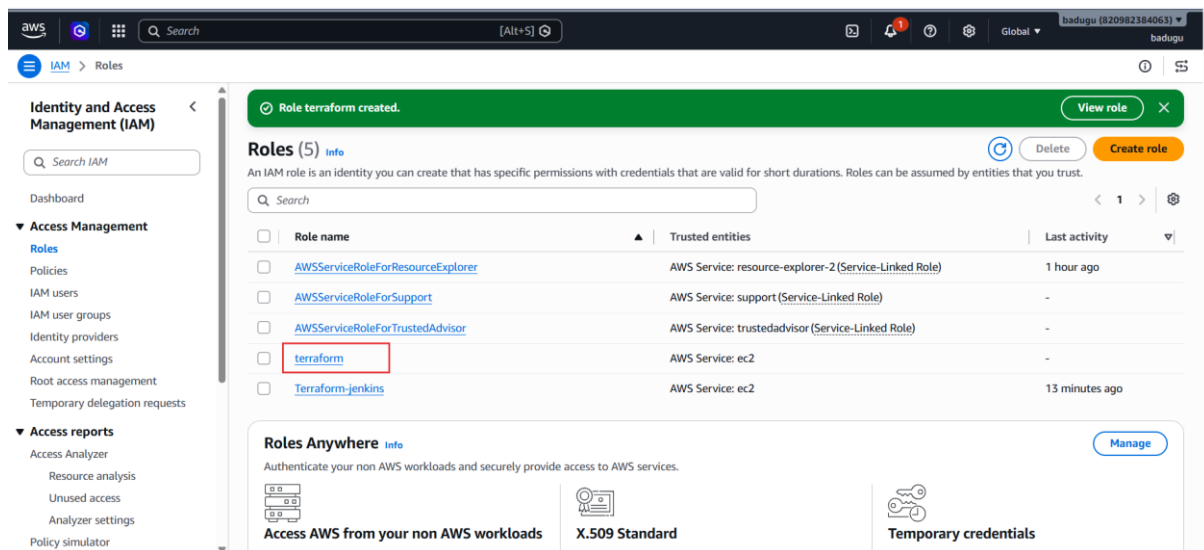
AWS resources

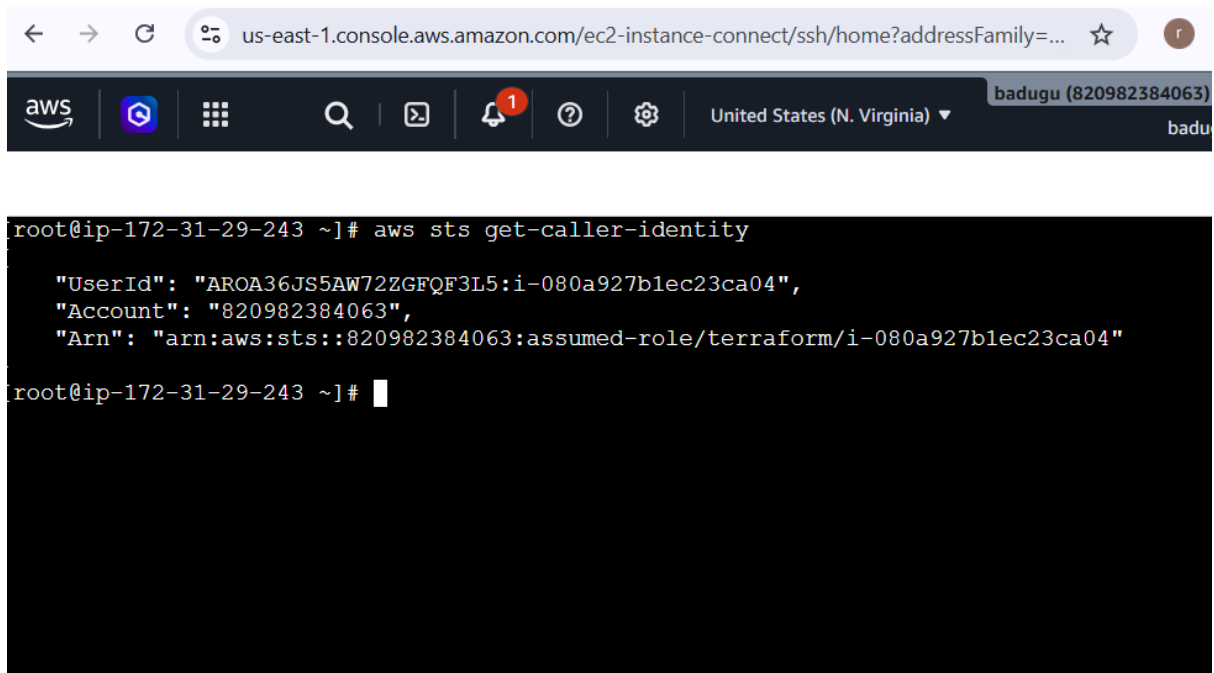
Then Terraform can use the EC2 instance role without putting AWS access keys inside Jenkins.

Test from Jenkins server:

`aws sts get-caller-identity`

You should get the AWS account and role information.





The screenshot shows the AWS Management Console interface. At the top, the browser address bar displays 'us-east-1.console.aws.amazon.com/ec2-instance-connect/ssh/home?addressFamily=...'. Below the address bar is the AWS navigation bar with the AWS logo, a search icon, a notification bell with a red '1', a help icon, a settings gear, and a dropdown menu for 'United States (N. Virginia)'. On the right side of the navigation bar, the user's name 'badugu (820982384063)' is visible. The main content area is a terminal window with a black background and white text. The terminal shows the command 'aws sts get-caller-identity' being executed, followed by the output: 'UserId': 'ARO36JS5AW72ZGFQF3L5:i-080a927b1ec23ca04', 'Account': '820982384063', 'Arn': 'arn:aws:sts::820982384063:assumed-role/terraform/i-080a927b1ec23ca04'. The terminal prompt is 'root@ip-172-31-29-243 ~]# '.

```
root@ip-172-31-29-243 ~]# aws sts get-caller-identity

  "UserId": "ARO36JS5AW72ZGFQF3L5:i-080a927b1ec23ca04",
  "Account": "820982384063",
  "Arn": "arn:aws:sts::820982384063:assumed-role/terraform/i-080a927b1ec23ca04"

root@ip-172-31-29-243 ~]#
```

6. Create Jenkins Pipeline

Create a new Jenkins job:

New Item



terraform-pipeline



Pipeline



OK

Select:

Pipeline script from SCM

Choose:

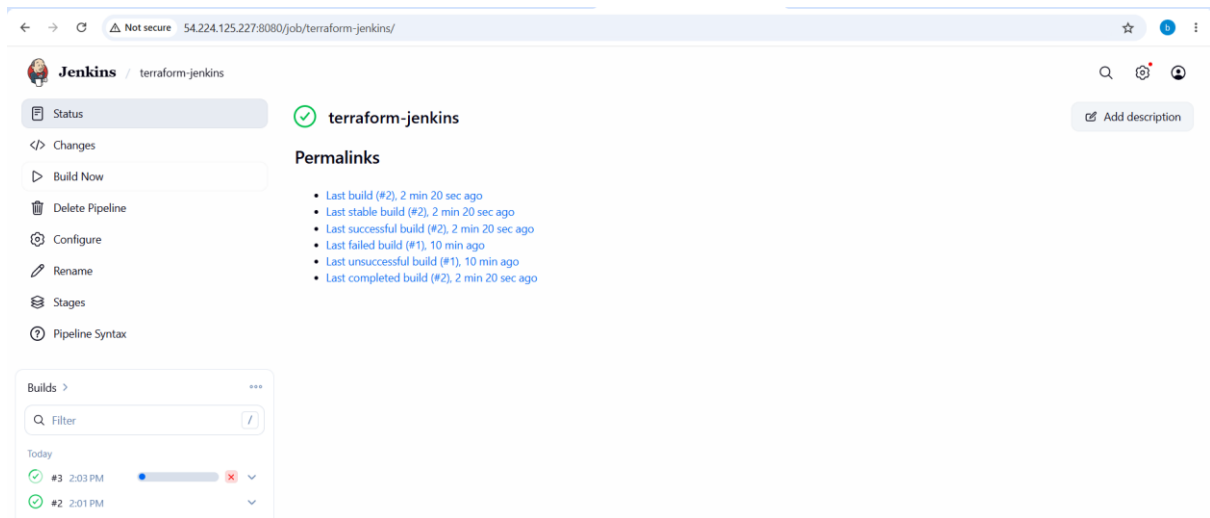
Git

Enter your GitHub repository URL.

For example:

<https://github.com/your-user/terraform-jenkins.git>

Select the branch:



*/main

Script Path:

Jenkinsfile

